



ARIENNE ALVES NAVARRO
CLARISSE LACERDA PIMENTEL
MARIA RITA RIBEIRO RESENDE
THALES RODRIGUES RESENDE

RELATÓRIO - ENTREGA 2
ANALISADOR SINTÁTICO (BISON)

1. GRAMÁTICA COMPLETA EM BNF

```
<programa> ::= <lista_comandos>

<lista_comandos> ::= <comando>
                    | <lista_comandos> <comando>

<comando> ::= <declaracao> SEMICOLON
            | <tipo> ID LPAREN <parametros> RPAREN <bloco>
            | <tipo> ID LPAREN RPAREN <bloco>
            | <atribuicao> SEMICOLON
            | <comando_if>
            | <comando_while>
            | <bloco>
            | <io_stmt>
            | <chamada_func> SEMICOLON
            | RETURN <expressao> SEMICOLON

<bloco> ::= LBRACE <lista_comandos> RBRACE
        | LBRACE RBRACE

<comando_if> ::= IF LPAREN <expressao> RPAREN <comando>
              | IF LPAREN <expressao> RPAREN <comando> ELSE <comando>

<comando_while> ::= WHILE LPAREN <expressao> RPAREN <comando>

<tipo> ::= INT
        | FLOAT

<declaracao> ::= <tipo> <lista_decl_itens>

<lista_decl_itens> ::= <decl_item>
                    | <lista_decl_itens> COMMA <decl_item>

<decl_item> ::= ID
              | ID ASSIGN <expressao>

<atribuicao> ::= ID ASSIGN <expressao>

<expressao> ::= <expressao> PLUS <expressao>
              | <expressao> MINUS <expressao>
              | <expressao> MULT <expressao>
              | <expressao> DIV <expressao>
              | <expressao> MOD <expressao>
              | <expressao> AND <expressao>
              | <expressao> OR <expressao>
              | <expressao> EQ <expressao>
              | <expressao> NE <expressao>
              | <expressao> LT <expressao>
              | <expressao> LE <expressao>
              | <expressao> GT <expressao>
              | <expressao> GE <expressao>
              | NOT <expressao>
              | MINUS <expressao>
              | LPAREN <expressao> RPAREN
              | NUM_INT
              | NUM_DEC
              | ID
```

```
| <chamada_func>

<parametros> ::= <parametro>
               | <parametros> COMMA <parametro>

<parametro> ::= <tipo> ID

<chamada_func> ::= ID LPAREN <argumentos> RPAREN
                | ID LPAREN RPAREN

<argumentos> ::= <expressao>
                | <argumentos> COMMA <expressao>

<io_stmt> ::= PRINT LPAREN <expressao> RPAREN SEMICOLON
            | READ LPAREN ID RPAREN SEMICOLON
```

2. DECISÕES DO PROJETO

Durante a concepção e implementação da Etapa 2 (Análise Sintática), o grupo adotou algumas estratégias principais para simplificar o desenvolvimento e garantir a robustez do compilador:

2.1. Otimização da Tabela de Símbolos (Evolução da Etapa 1)

Atendendo às recomendações da etapa léxica, a Tabela de Símbolos foi refatorada de um vetor estático de busca linear para uma tabela de dispersão (*hashmap*). A resolução de colisões foi implementada via encadeamento separado. Essa melhoria reduz a complexidade temporal de busca e inserção para uma média de $O(1)$, otimizando a integração entre o *lexer* e o *parser*.

2.2. Organização da linguagem em comandos e blocos

A linguagem foi centralizada na abstração de comando, permitindo que estruturas de controle condicionais e de repetição aceitem tanto instruções isoladas quanto blocos delimitados por chaves (`{}`). Essa decisão aproxima o comportamento da linguagem ao encontrado em linguagens imperativas tradicionais e aumenta a flexibilidade sintática.

2.3. Consolidação das expressões

Evitou-se a fragmentação recursiva de expressões matemáticas e lógicas em múltiplos não-terminais (como regras de termo e fator). Todas as operações foram unificadas no não-terminal *expressao*, delegando a resolução de precedência e associatividade nativamente ao Bison (via diretivas `%left` e `%precedence`). Essa abordagem eliminou *warnings* de associatividade inútil e reduziu significativamente a complexidade de manutenção do *parser*.

2.4. Tratamento e reporte de erro

Implementou-se a recuperação de erros sintáticos utilizando a estratégia de *Panic Mode*, adotando o ponto e vírgula (`;`) como *token* de sincronização. Isso permite que o compilador ignore os *tokens* problemáticos até encontrar o delimitador, conseguindo reportar múltiplos erros em uma única execução. Além disso, o parser indica a linha e a coluna exatas da falha, baseando-se nas informações de posicionamento capturadas pelo analisador léxico.

3. DIFICULDADES ENCONTRADAS

A principal dificuldade durante a modelagem da gramática livre de contexto residiu no tratamento de ambiguidades inerentes à sintaxe de linguagens *C-like*. Durante as compilações iniciais, o Bison reportou conflitos de *Shift/Reduce* que precisaram ser tratados pontualmente:

3.1. Conflito do Dangling Else

Durante o desenvolvimento, o Bison acusou um conflito *shift/reduce* clássico referente ao *dangling else*. O analisador entrava em impasse por não conseguir decidir se deveria reduzir um comando *if* incompleto ou empilhar (*shift*) o *token* ELSE. O problema foi solucionado com a criação de diretivas de precedência fictícias no cabeçalho (%precedence LOWER_THAN_ELSE e %precedence ELSE). Ao aplicar a menor precedência (%prec LOWER_THAN_ELSE) diretamente na regra do *if* simples, forçamos o autômato a priorizar o deslocamento, associando corretamente o *else* ao *if* mais próximo e eliminando o conflito.

3.2. Conflito do Menos Unário (Unary Minus)

Outra dificuldade foi a ambiguidade estrutural entre o operador de subtração binária e a inversão de sinal (menos unário), já que ambos compartilham o mesmo símbolo léxico (MINUS). Para evitar que o Bison aplicasse a prioridade da subtração padrão, foi criado um *token* virtual de precedência máxima (%precedence UMINUS). Esse identificador foi então associado exclusivamente à regra de inversão de sinal por meio do modificador contextual %prec UMINUS, garantindo que a operação unária seja resolvida antes de qualquer outra operação aritmética binária.

4. CONJUNTOS FIRST E FOLLOW

Como o Bison utiliza um *parser bottom-up* LALR(1), regras com recursão à esquerda podem ser utilizadas diretamente, sem necessidade de refatoração da gramática. As ambiguidades relacionadas à precedência e associatividade de operadores foram resolvidas por meio das diretivas %left, %precedence e %prec.

Isso dispensa transformações teóricas como estratificação ou eliminação de recursão. Assim, a análise foca apenas no não-terminal raiz Expressao, cuja produção generalizada é:

$$\text{Expressao} \rightarrow \text{Expressao OP Expressao} \mid (\text{Expressao}) \mid \text{OP}_{\text{unario}} \text{Expressao} \mid \text{ID} \mid \text{NUM}$$

Onde:

- OP: Conjunto de todos os operadores binários suportados (PLUS, MINUS, MULT, DIV, MOD, AND, OR, EQ, NE, LT, LE, GT, GE).
- OP_{unario}: Operadores que afetam um único operando à direita (NOT e o menos unário MINUS).
- ID e NUM: Identificadores de variáveis e literais numéricos base (NUM_INT, NUM_DEC).
- (Expressao): Representa o aninhamento por parênteses (LPAREN e RPAREN).

As tabelas a seguir detalham o mapeamento estrutural dos conjuntos FIRST e FOLLOW obtidos:

Não-terminal	FIRST
Expressao	{ LPAREN, NOT, MINUS, ID, NUM_INT, NUM_DEC }

Não-terminal	FOLLOW ¹
Expressao	{ \$, RPAREN, SEMICOLON, COMMA, PLUS, MINUS, MULT, DIV, MOD, AND, OR, EQ, NE, LT, LE, GT, GE }

¹ Como a análise foi delimitada estritamente aos não-terminais de expressões, assumimos temporariamente o não-terminal Expressao como o símbolo inicial deste subconjunto gramatical.

5. ESTADOS DO AUTÔMATO COM CONFLITOS

A gramática final não apresenta conflitos, mas ambiguidades clássicas precisaram ser tratadas para evitar problemas no autômato gerado pelo Bison.

5.1. Onde os conflitos poderiam aparecer

Se não tratados, os conflitos *shift/reduce* surgiriam em dois pontos: nas expressões matemáticas (devido à falta de precedência e associatividade explícitas, ex: $A + B * C$) e no clássico problema do *dangling else* (onde o analisador não saberia a qual *if* associar um bloco *else* solto).

5.2. Decisão de projeto

Em vez de reescrever a gramática quebrando as regras em múltiplos níveis complexos (o que dificultaria a leitura), optamos por manter a gramática limpa e plana. A decisão foi delegar a resolução das ambiguidades diretamente ao motor do Bison, utilizando diretivas de precedência e associatividade.

5.3. Como as declarações resolveram o problema

- **Expressões:** Utilizamos diretivas como `%left` e `%precedence` para definir a associatividade à esquerda e organizar a hierarquia dos operadores (do menor lógico OR ao maior unário `%prec UMINUS`).
- **Dangling Else:** Declaramos níveis de precedência fictícios (`%precedence LOWER_THAN_ELSE` seguido de `%precedence ELSE`). Ao aplicar `%prec LOWER_THAN_ELSE` na regra do *if* simples, garantimos que o *token* ELSE tenha prioridade. Assim, em caso de ambiguidade, o Bison força o *shift* (empilhamento), associando o *else* corretamente ao *if* mais próximo.

5.4. Evidência de compilação sem conflitos

A aplicação correta dessas diretivas tornou o autômato determinístico. Ao compilar o arquivo com `bison -d -v parser.y`, o relatório gerado no arquivo `parser.output` confirma explicitamente a existência de 0 conflitos *shift/reduce* e 0 conflitos *reduce/reduce*.

6. ARQUIVO DE TESTE E SUA SAÍDA

Para validar a robustez do analisador sintático e a eficácia das regras de recuperação de erro, elaborou-se um arquivo de código mesclando estruturas válidas da linguagem (declarações de escopo, desvios de fluxo, operações lógicas e matemáticas) com erros léxicos e sintáticos intencionais.

- **Arquivo de Entrada**

```
/*  
comentário inútil  
*/  
  
int a;  
int b, c;  
int flag;  
int ativo, concluido;  
  
int z  
  
a = 5;  
b = a;  
c = a + b * 3;
```

```
flag = 1;
ativo = 0;
concluido = ativo;
```

```
int w,;
```

```
a = 3 + 2 * 5;
b = (a + 1) * 4 - 2;
c = -a + (b - -c) / 2;
```

```
a = ;
```

```
flag = a < b;
flag = a <= b;
flag = a > b;
flag = a >= b;
flag = a == b;
flag = a != b;
```

```
if (a < b
    c = 1;
```

```
flag = 1 && 0;
flag = (a < b) || (c == a);
flag = !flag;
flag = !(a >= b && c != a);
```

```
if (a >= 10 && a <= 20 || !(c == 5))
    a = a + 1;
```

```
while ()
    a = a + 1;
```

```
@
```

```
int S = -2;
P = -0;
```

```
if (a < b)
    a = b;
```

```
if (a == b)
    c = 1;
else
    c = 2;
```

```
a = 5 +* 2;
```

```
//dangling else
if (a < b)
    if (b < c)
        a = b;
    else
        c = a;
```

```
while (a < 10)
    a = a + 1;
```

```

while (a < 20) {
    a = a + 1;
    print(a);
}

print(a);
read(b);

{
    int x, y;
    x = 10;
    y = 20;
    print(x);
    print(y);
}

else
    a = 1;

```

- **Saída**

ERRO SINTATICO: syntax error na Linha 12, Coluna 1
 ERRO SINTATICO: syntax error na Linha 19, Coluna 7
 ERRO SINTATICO: syntax error na Linha 25, Coluna 5
 ERRO SINTATICO: syntax error na Linha 35, Coluna 5
 ERRO SINTATICO: syntax error na Linha 45, Coluna 8

ERRO LEXICO: '@' na Linha 48, Coluna 1
 ERRO SINTATICO: syntax error na Linha 61, Coluna 8
 ERRO SINTATICO: syntax error na Linha 89, Coluna 1

Analise sintatica concluida.

=== TABELA DE SIMBOLOS ===

Identificador	Linha	Coluna

P	51	1
S	50	5
a	5	5
b	6	5
c	6	8
w	19	5
x	82	9
y	82	12
z	10	5
concluido	8	12
flag	7	5
ativo	8	5